

Infografía

Módulo 7 — Godot 4 • Sesión 4

Animaciones y máquinas de estado

UPB • Gestión I/2026

Plan de la clase

1. **AnimationPlayer** — anima *cualquier* propiedad (el flipbook y más).
2. **Máquinas de estado finitas** — qué son, en general.
3. **AnimationTree** + **BlendSpace2D** — la dirección sin 12 **if**.
4. **Dos máquinas de estado** — el "aha" del día.
5. **Arena de combate** — el ataque activa el golpe *desde la animación*.
6. **Ejercicio** — darle al enemigo su propia máquina de estado.

Sesión 2: movimos un vector. Sesión 3: el mundo tiene física. Hoy: cómo se VE todo eso — y cómo el código y lo visual se mantienen sincronizados.

La pregunta del día

Cuando el personaje corre a la derecha,
¿**quién** decide qué imagen se muestra?

No tu código de movimiento. Esa es toda la idea.

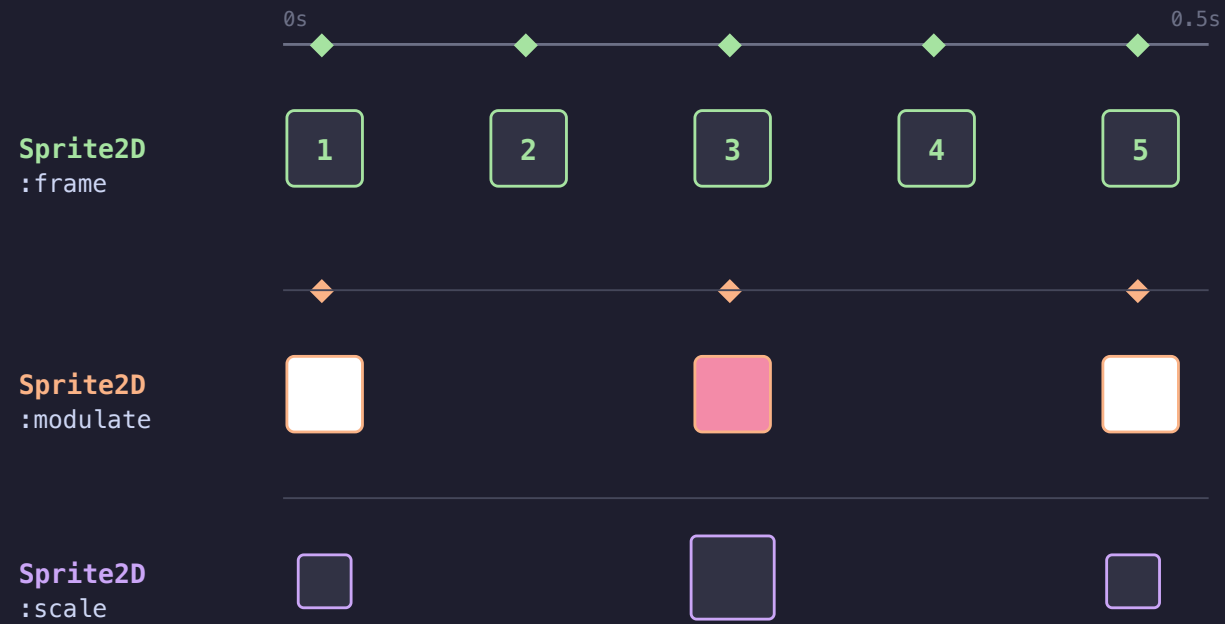
Parte 1 — `AnimationPlayer`

Una **animación** es una lista de **propiedades que cambian en el tiempo**.

- `Sprite2D:frame` $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \dots$ = el clásico **flipbook**.
- Pero NO es solo el frame: podés animar `modulate`, `scale`, `position`, `disabled` ... cualquier propiedad de cualquier nodo.

Pista clave: cada animación tiene una pista `RESET` — el estado "neutro" al que vuelve.

una animación = propiedades que cambian en el tiempo



la MISMA herramienta anima cualquier propiedad — no solo el frame

Demo — 01_animationplayer.tscn

- `caminar` → anima `Sprite2D:frame` (flipbook).
- `rebote` (ESPACIO) → anima `scale` + `modulate`.

La misma herramienta, propiedades distintas.

```
func _ready():  
    anim.play("caminar")  
  
func _unhandled_input(e):  
    if e.is_action_pressed("attack"):  
        anim.play("rebote")
```

Máquinas de estado finitas (FSM)

Antes de Godot, la idea — y es vieja y universal.

Una **FSM** modela algo que está en **uno** de un número **finito** de estados, y solo cambia de estado por **transiciones** bien definidas.

Un semáforo, una puerta, un cajero, un personaje de juego: todos son FSM.

una máquina de estado está en UN estado a la vez; cambia solo por transiciones definidas



finita = lista cerrada de estados · solo se permiten las transiciones dibujadas

de Verde NO podés saltar directo a Rojo: no hay flecha

Conceptos clave

- **Estado** — un modo en el que el sistema puede estar (`Verde`, `idle`).
- **Transición** — el paso de un estado a otro (la flecha).
- **Evento / condición** — el disparador que activa una transición.
- **Estado inicial** — en cuál arranca.
- **Una sola activa** — nunca estás en dos estados a la vez.
- **Transiciones definidas** — solo valen las flechas que dibujaste; no se salta de cualquier estado a cualquier otro.

Opcional: acciones de entrada y salida (qué hacer al entrar/salir de un estado).

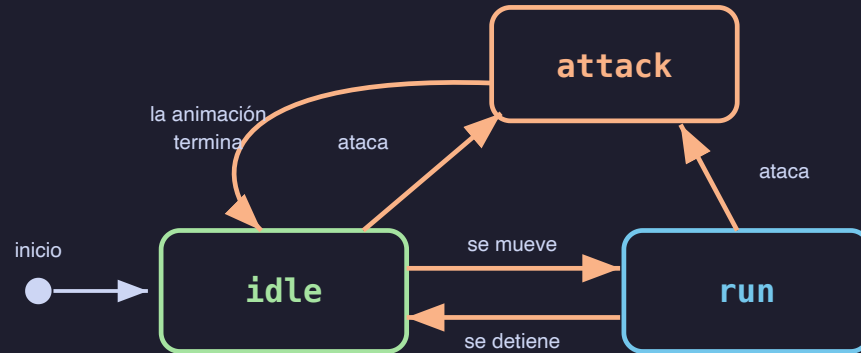
¿Por qué nos sirve?

Sin FSM, el comportamiento se vuelve una maraña de banderas:

```
if not attacking and not hurt and moving and not dead:  
    ...    # ¿y si está atacando Y herido? 🤖
```

Con FSM: **un** estado a la vez, transiciones claras. El código se lee como el diagrama.

lo mismo, aplicado a un personaje — es la *StateMachine* del *AnimationTree*



cada flecha = una transición

el texto sobre la flecha = el evento que la dispara
el personaje está SIEMPRE en exactamente un estado

en código:

```
state_machine  
.travel(  
"run")
```

= disparar una transición

Una máquina de estado mínima

Un personaje tiene **estados**: `idle`, `run`, `attack`. Cada uno, una animación.

El `AnimationTree` trae una `StateMachine`: vos pedís `travel("run")` y él hace la transición. El código decide *cuándo*; el árbol decide *qué se ve*.

```
if input_vector != Vector2.ZERO:  
    state_machine.travel("run")  
else:  
    state_machine.travel("idle")
```

Todavía SIN direcciones: el personaje mira siempre hacia abajo. Una cosa a la vez.

Demo — 02_estado_simple.tscn

- `AnimationTree` con una `StateMachine`:
`idle` / `run` / `attack`.
- Cada estado es **una** animación (no hay `BlendSpace` todavía).
- `attack` se reproduce una vez y vuelve sola (pista de método).

```
enum { IDLE, RUN, ATTACK }  
var state = IDLE  
  
func attack_anim_finished():  
    state = IDLE
```

Misma estructura que la arena — pero mínima.

El problema

Ya tenemos `idle` / `run` / `attack`. Ahora quiero que el personaje mire **hacia donde camina** — 4 direcciones × 3 estados.

Eso son $4 \times 3 = 12$ animaciones. ¿Las elegís con `if`?

```
if state == RUN:
    if dir.x > 0: play("RunRight")
    elif dir.x < 0: play("RunLeft")
    elif dir.y > 0: play("RunDown")
    ...    # y otra vez para Idle, y otra para Attack 🤖
```

Hay una herramienta para esto.

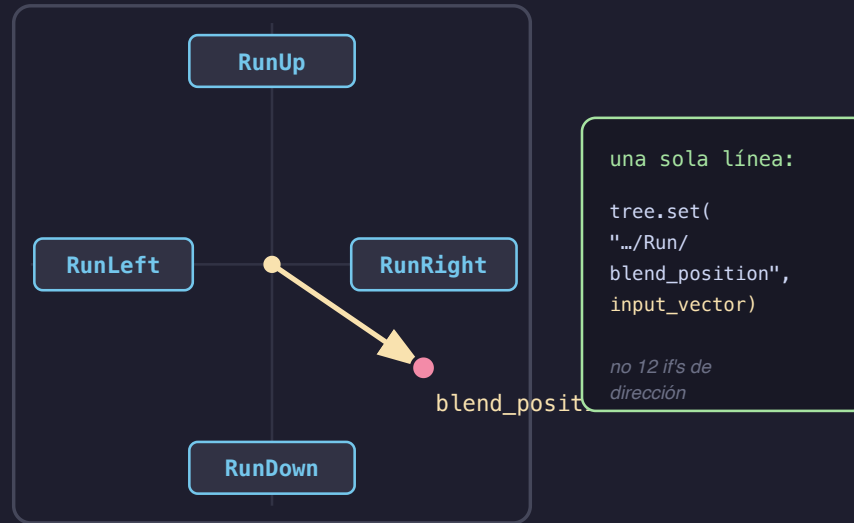
AnimationTree + BlendSpace2D

El **AnimationTree** es una máquina de estado *de animaciones*.

- Cada estado (**Idle**, **Run**, **Attack**) es un **BlendSpace2D**.
- Un **BlendSpace2D** coloca las 4 direcciones en un plano.
- Le pasás **un vector** (**blend_position**) y él elige/mezcla.

```
tree.set("parameters/Run/blend_position", input_vector)  
state_machine.travel("Run")
```

BlendSpace2D — un solo vector elige la dirección de la animación



le pasás el vector de input — el árbol mezcla/elige la animación correcta

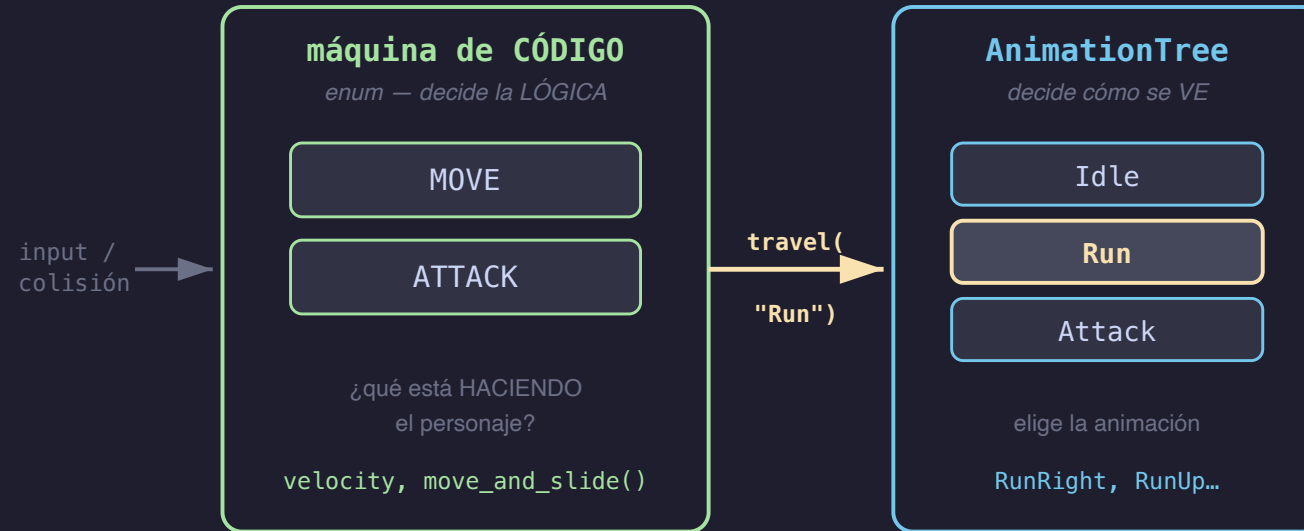
El "aha" del día

Hay **dos** máquinas de estado.

- Una en **código** (`enum`) — decide la **lógica**.
- El `AnimationTree` — decide cómo se **ve**.

Tu código nunca dice "frame 14". Dice "estoy en Run" y viaja: `travel("Run")`.

dos máquinas de estado, dos responsabilidades distintas



el código nunca dice "mostrá el frame 14"
dice "estoy corriendo a la derecha" y el árbol resuelve el resto

Demo — 03_arena_combate.tscn

La máquina de código del player:

```
enum { MOVE, ATTACK }  
var state = MOVE  
  
func _physics_process(delta):  
    match state:  
        MOVE: move_state(delta)    # input → blend_position + travel  
        ATTACK: attack_state()      # travel("Attack")
```

Completamos los `travel()` y el `blend_position` en vivo.

El ataque que se activa solo

El golpe (`HitBox`) **no** se prende con código suelto.
Lo prende la **propia animación de ataque**:

- una pista que pone `HitBox/CollisionShape2D:disabled` en `false` justo en el frame del golpe, y en `true` al terminar.
- una **pista de método** que llama `attack_anim_finished()` al final.

La animación maneja el timing del gameplay. Eso conecta con las capas/máscaras de la sesión 3: el `HitBox` vive en `player_hit`.

El componente **Health**

No sabe de animaciones ni de input. Solo cuenta y **avisa con señales**.

```
signal health_changed(old, new)
signal health_depleted

func take_damage(hit: HitBox):
    health -= hit.damage
    health_changed.emit(...)
    if health == 0: health_depleted.emit()
```

El dueño (player o enemigo) decide QUÉ hacer al recibir la señal: parpadear, morir, actualizar el HUD...

Break 

5 minutos

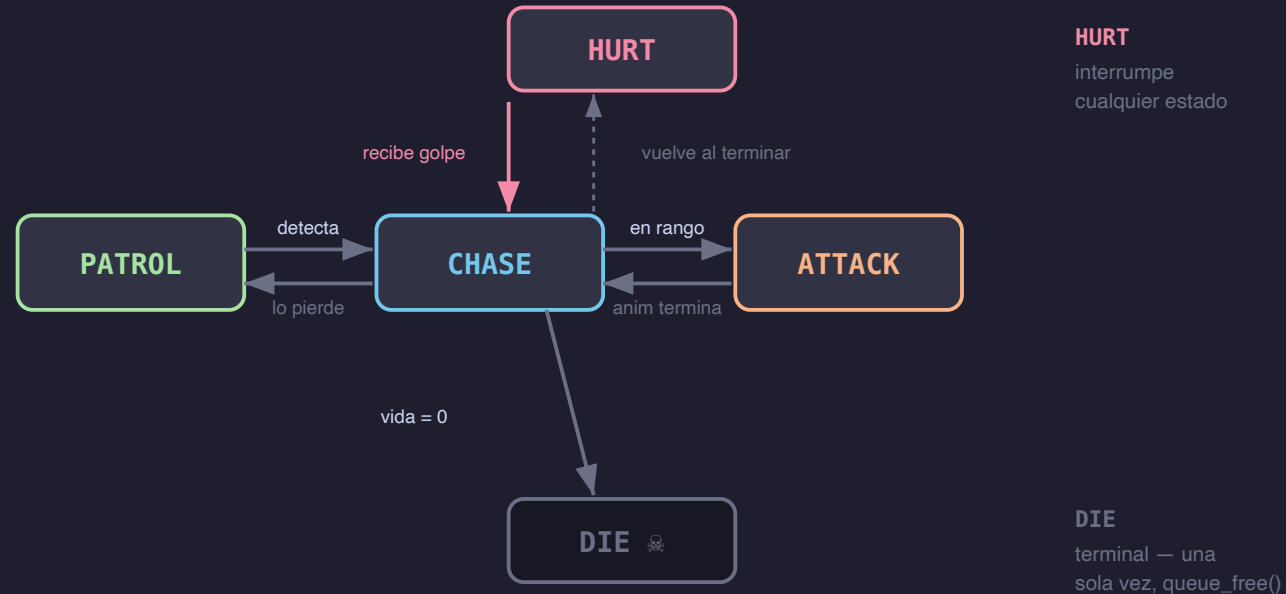
Ejercicio — la máquina del enemigo

`04_enemigo_estados.tscn`. El murciélago ya tiene su `AnimationTree` y sus animaciones (`fly` / `attack` / `hurt` / `die`). **Falta la máquina de código.**

Cinco estados:

`PATROL` se pasea · `CHASE` persigue · `ATTACK` golpea · `HURT` reacciona · `DIE` muere

la máquina de estado del enemigo — el ejercicio



cada flecha es una transición — las señales (detección, daño) las disparan

Lo que tenés que escribir

1. `_set_state()` — el puente: cambia el `enum` y llama a `travel(...)`.
2. `_patrol()` / `_chase()` — el movimiento de cada estado.
3. Señales del `DetectionArea` — entrar a `CHASE`, volver a `PATROL`.
4. `_on_hurt_box_area_entered()` — recibir daño.
5. Señales de `Health` — `HURT` si queda vida, `DIE` si llegó a 0.
6. Las pistas de método — `attack_anim_finished()`, `hurt_anim_finished()`, `die_anim_finished()` (esta hace `queue_free()`).

La solución de referencia la tiene el docente.

Pistas

- `HURT` **interrumpe**: podés entrar desde cualquier estado.
- `DIE` es **terminal**: una sola vez, y al terminar la animación se borra.
- `_set_state()` debe ser el **único** lugar que llama `travel()`.
Si lo dispersás, vas a pelear con vos mismo.

El murciélago tiene UNA sola animación de vuelo. Los estados se distinguen por `modulate` (flash rojo), `scale` (lunge), `alpha` (fade)... otra vez: anima cualquier propiedad.

Resumen

- `AnimationPlayer` anima *cualquier* propiedad en el tiempo.
- `AnimationTree` es una máquina de estado de animaciones; `BlendSpace2D` elige la dirección con un vector.
- Hay **dos máquinas**: el `enum` (lógica) y el árbol (visual). `travel()` las sincroniza.
- Una animación puede **manejar gameplay** (activar un HitBox) con pistas de valor y de método.
- `Health` + señales = el patrón para "reaccionar a que algo pasó".

El código dice qué HACÉS; el árbol dice cómo te VES.

La próxima

Mundo (TileMapLayer) y UI

Terminá la máquina del enemigo — la vas a reusar.